BÁO CÁO TỔNG HỢP KIỂM THỬ MIỀN VÀ PHÂN TÍCH GIÁ TRỊ BIÊN (HW02)

HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ ESHOP (SUT)

THÔNG TIN HỌC VIÊN & BÀI NỘP

- **Môn học:** Kiểm thử phần mềm nâng cao (Advanced Software Testing / KCPM)
- **Mã lớp:** CS423 / CSC15003 - FIT @ HCMUS
- **Bài tập:** HW02 – Domain Testing on EShop
- **Thành viên:** Thành viên 5 (Member 5)
- **Họ và tên:** Nguyễn Bảo An *(Đại diện nhóm phân công Member 5)*
- **Mã số sinh viên (MSSV):** 23120005
- **Ngày hoàn thành:** 15/10/2026
- **Kho mã nguồn SUT:** https://github.com/ttbhanh/eshop-sut
- **Cấp độ Bloom-AI cam kết:** G9.2 (Áp dụng - Apply) & G9.3 (Phân tích - Analyse)

MỤC LỤC

1. Giới Thiệu Tổng Quan & Phạm Vi Phân Công
2. Phương Pháp Luận Kiểm Thử (Testing Methodology)
    - 2.1 Phân vùng tương đương (Equivalence Partitioning - EP)
    - 2.2 Phân tích giá trị biên (Boundary Value Analysis - 2-point & 3-point BVA)
    - 2.3 Chiến lược ưu tiên AI có kiểm soát (AI-First & Human-in-the-Loop)
3. Tính Năng A (FR-06): Xem Chi Tiết Sản Phẩm & Thêm Vào Giỏ
    - 3.1 Phân tích miền biến & Không gian trạng thái
    - 3.2 Ma trận kiểm thử miền & Phân tích giá trị biên
    - 3.3 Kết quả thực thi kiểm thử & Xác nhận lỗi thực tế
    - 3.4 Phân tích khoảng trống AI (AI Gap Analysis)
4. Tính Năng B (FR-11): Lịch Sử Đơn Hàng & Hủy Đơn Hàng
    - 4.1 Phân tích miền biến & Máy trạng thái đơn hàng (Order State Machine)
    - 4.2 Ma trận kiểm thử miền & Phân tích giá trị biên
    - 4.3 Kết quả thực thi kiểm thử & Xác nhận lỗi thực tế
    - 4.4 Phân tích khoảng trống AI (AI Gap Analysis)
5. Tính Năng C (FR-19): Quản Lý Người Dùng Dành Cho Admin
    - 5.1 Phân tích miền biến, Kiểm soát quyền & Ràng buộc toàn vẹn
    - 5.2 Ma trận kiểm thử miền & Phân tích giá trị biên
    - 5.3 Kết quả thực thi kiểm thử & Xác nhận lỗ hổng bảo mật (BOLA / IDOR)
    - 5.4 Phân tích khoảng trống AI (AI Gap Analysis)
6. Tính Năng D (Mobile FR-10): Hủy Đơn Hàng Trên Ứng Dụng Di Động
    - 6.1 Phân tích miền hiển thị UI, UX Touchscreen & Tương tác API
    - 6.2 Ma trận kiểm thử miền & Phân tích giá trị biên
    - 6.3 Kết quả thực thi kiểm thử & Độ lệch hợp đồng (Contract Divergence)
    - 6.4 Phân tích khoảng trống AI (AI Gap Analysis)
7. Báo Cáo Tổng Hợp Các Lỗi Phần Mềm Đã Phát Hiện (Consolidated Bug Report)
8. Tổng Hợp Phân Tích Khoảng Trống Của AI (AI Gap Analysis Synthesis)
9. Bộ Kỹ Năng Agent Tự Động (Agent Skills: domain-bva-generator)
10. Phụ Lục Bắt Buộc (Mandatory Appendices)
    - Phụ lục 1: Phê bình / Đánh giá AI (AI Critique)
    - Phụ lục 2: Báo cáo kiểm toán sử dụng AI (AI Audit Report)

1. Giới Thiệu Tổng Quan & Phạm Vi Phân Công

Hệ thống Thương mại Điện tử EShop là một ứng dụng full-stack hoàn chỉnh bao gồm: - **Backend API Server:** Node.js, Express REST API, SQLite3 (`backend/server.js`). - **Web Frontend:** React 18, Vite, React Router, TailwindCSS (`frontend-web/`). - **Web Admin Portal:** React 18, Vite, TailwindCSS (`frontend-admin/`). - **Mobile Client:** React Native (`frontend-mobile/App.js`).

Theo bảng phân công nhóm (Mục 5 SRS & `group_feature_assignment.md`), **Thành viên 5** phụ trách 4 tính năng độc lập, đại diện cho đầy đủ 4 nhóm phân loại nghiệp vụ của hệ thống:

Nhóm nghiệp vụ	Mã tính năng	Tên tính năng	Mã nguồn SUT liên quan
Pool A (Xác thực & SP)	**FR-06**	Chi tiết sản phẩm & Thêm vào giỏ hàng	`frontend-web/src/pages/ProductDetai` (`GET /api/products/:id`)
Pool B (Giỏ & Đơn hàng)	**FR-11**	Lịch sử đơn hàng của người dùng	`frontend-web/src/pages/OrderHistory` (`GET/PUT /api/orders`)
Pool C (Web Admin)	**FR-19**	Quản lý người dùng dành cho Admin	`frontend-admin/src/pages/Users.jsx` (`GET/DELETE /api/admin/users`)
Pool D (Mobile App)	**Mobile FR-10**	Hủy đơn hàng trên ứng dụng di động	`frontend-mobile/App.js` (`cancelOrder`)

Mục tiêu cốt lõi của đợt kiểm thử này là áp dụng nghiêm ngặt các nguyên lý **Kiểm thử Miền (Domain Testing)** và **Phân tích Giá trị Biên (Boundary Value Analysis)** theo chuẩn công nghiệp ISTQB, kết hợp khai thác sức mạnh của AI Agent một cách có kỷ luật (Human-in-the-Loop) để rà quét triệt để các góc khuất, logic bẫy mã nguồn (seeded bugs), lỗi phân quyền (Broken Object Level Authorization - BOLA), và các bất cập về trải nghiệm người dùng (UX).

2. Phương Pháp Luận Kiểm Thử (Testing Methodology)

2.1 Phân vùng tương đương (Equivalence Partitioning - EP)

Kỹ thuật phân vùng tương đương chia toàn bộ miền giá trị đầu vào của một trường hoặc tham số thành các tập con rời rạc (partitions), với giả định rằng mọi giá trị trong cùng một phân vùng sẽ được hệ thống xử lý hoàn toàn như nhau: - **Phân vùng Hợp lệ (Valid Equivalence Partition):** Tập hợp các giá trị được hệ thống chấp nhận theo đúng đặc tả nghiệp vụ SRS. - **Phân vùng Không hợp lệ (Invalid Equivalence Partition):** Tập hợp các giá trị vi phạm ràng buộc về kiểu dữ liệu, định dạng, độ dài, hoặc giới hạn logic mà hệ thống bắt buộc phải từ chối hoặc trả về thông báo lỗi thích hợp.

2.2 Phân tích giá trị biên (Boundary Value Analysis - BVA)

Kinh nghiệm kiểm thử phần mềm thực tế chứng minh phần lớn lỗi tiềm ẩn tập trung tại các đường biên ngăn cách giữa các phân vùng tương đương. Chúng tôi triển khai kết hợp cả hai kỹ thuật biên chuẩn: 1. **2-point BVA:** Với mỗi biên, khảo sát 2 điểm: - **On-point:** Giá trị nằm chính xác ngay trên đường ranh giới. - **Off-point:** Giá trị nằm ngay sát cạnh đường ranh giới nhưng thuộc về phía đối diện (ngoài vùng hợp lệ). 2. **3-point BVA (Extended Boundary Testing):** Khảo sát 3 điểm gồm $\{Bin - 1, Bin, Bin + 1\}$ (hoặc On, In, Out) nhằm bảo đảm không xảy ra lỗi lệch 1 (Off-by-one errors: < thay vì <=, > thay vì >=). 3. **Phân tích biên chuyển trạng thái (State Transition Boundaries):** Khảo sát các điểm trạng thái hợp lệ và bất hợp lệ khi kích hoạt hành động nghiệp vụ (ví dụ: chuyển đổi trạng thái đơn hàng).

2.3 Chiến lược ưu tiên AI có kiểm soát (AI-First & Human-in-the-Loop)

- Thay vì để AI làm việc như một "chiếc hộp đen" đưa ra các test case chung chung, chúng tôi triển khai quy trình 4 giai đoạn chuẩn:
  1. *Prompting định hướng:* Yêu cầu AI phân tích mã nguồn SUT, trích xuất biến và phân loại vùng tương đương.
  2. *Thiết kế ma trận BVA:* Dùng AI hỗ trợ tổng hợp bảng ma trận test case On/Off/In/Out.
  3. *Thực thi & Đối soát độc lập:* Chạy test script thực tế trên SUT để xác minh phản hồi thực.
  4. *AI Gap Analysis:* Rà soát những điểm AI bị ảo giác (hallucination) hoặc thiên kiến (happy-path bias) để con người tự bổ sung các ca kiểm thử bảo mật chuyên sâu.

---

3. Tính Năng A (FR-06): Xem Chi Tiết Sản Phẩm & Thêm Vào Giỏ

3.1 Phân tích miền biến & Không gian trạng thái

- **Đầu vào 1:** `quantity` **(Số lượng đặt mua):**
  - Miền hợp lệ: Tập số nguyên $Q \in \mathbb{Z}^+, 1 \leq Q \leq$ Stock.
  - Miền không hợp lệ: $Q \leq 0$ (số âm, số 0), số thực không nguyên (1.5), giá trị phi số (`abc`, `null`, `NaN`).
  - Điểm biên khảo sát:
    - Biên dưới: 0 (Off-point, Invalid), 1 (On-point, Valid), 2 (In-point, Valid).
    - Biên trên: Stock (On-point, Valid), Stock $+ 1$ (Off-point, Invalid).
- **Đầu vào 2:** `clickCount` **(Trạng thái số lần bấm nút "Thêm vào giỏ hàng"):**
  - Biến trạng thái nội bộ React `clickCount`: 0 hoặc 1.
  - Hành vi mong đợi: Bấm 1 lần là thêm vào giỏ.

3.2 Ma trận kiểm thử miền & Phân tích giá trị biên (12 Test Cases)

Mã TC	Hạng mục kiểm thử	Giá trị đầu vào	Kỳ vọng (SRS Standard)	Kết quả thực tế trên SUT	Kết luận
TC-FR06-01	BVA Biên dưới On-point	`quantity = 1`	Thêm thành công 1 SP vào giỏ	Thêm thành công (nhưng cần 2 lần bấm)	**PASS / CONDITIONAL**
TC-FR06-02	BVA Biên dưới In-point	`quantity = 2`	Thêm thành công 2 SP vào giỏ	Thêm thành công 2 SP	**PASS**
TC-FR06-03	BVA Biên dưới Off-point	`quantity = 0`	Chặn lại, báo lỗi số lượng ≥ 1	**SUT chấp nhận quantity = 0!**	❌ **FAIL (BUG-02)**
TC-FR06-04	Domain Số âm	`quantity = -1`	Chặn lại, báo lỗi số lượng âm	**SUT chấp nhận quantity = -1!**	❌ **FAIL (BUG-02)**
TC-FR06-05	Domain Số âm sâu	`quantity = -100`	Chặn lại, báo lỗi số lượng âm	**SUT chấp nhận quantity = -100!**	❌ **FAIL (BUG-02)**
TC-FR06-06	Domain Số thực	`quantity = 1.5`	Chặn số thập phân hoặc làm tròn	Input HTML `step` không chặn triệt để	❌ **FAIL (Cảnh báo)**
TC-FR06-07	Domain Chuỗi phi số	`quantity = "abc"`	Chặn chuỗi hoặc fallback về 1	Giỏ hàng lưu `quantity = NaN`	❌ **FAIL (BUG-02)**

Mã TC	Hạng mục kiểm thử	Giá trị đầu vào	Kỳ vọng (SRS Standard)	Kết quả thực tế trên SUT	Kết luận
TC-FR06-08	UX First Click Trap	Bấm nút lần 1	Thêm ngay vào giỏ và hiện toast	**Bị nuốt sự kiện, không phản hồi**	**FAIL (BUG-01)**
TC-FR06-09	UX Second Click	Bấm nút lần 2	Thêm vào giỏ và hiện nút "Đã thêm"	Thêm vào giỏ và đổi chữ nút	**PASS**
TC-FR06-10	API Backend ID chẵn	`GET /api/products/2`	Trả về `price` dạng `number`	**Trả về `price` dạng `string` ("28000000")**	**FAIL (BUG-01 Backend)**
TC-FR06-11	API Backend ID lẻ	`GET /api/products/1`	Trả về `price` dạng `number`	Trả về `price` dạng `number`	**PASS**
TC-FR06-12	API ID không tồn tại	`GET /api/products/9999`	Trả về HTTP 404 Not Found	Trả về HTTP 200 kèm `{}`	**FAIL (REST Standard Bug)**

3.3 Kết quả thực thi kiểm thử & Xác nhận lỗi thực tế

Kịch bản tự động `node tests/verify_fr06.js` đã xác minh thành công 100%: 1. **Lỗi UX nuốt sự kiện bấm nút (BUG-01):** Tại `frontend-web/src/pages/ProductDetail.jsx` (dòng 23-28), mã nguồn cài bẫy: `javascript if (clickCount === 0) { setClickCount(1); return; // Lần bấm đầu tiên bị dừng ngay lập tức, không gọi addToCart()! } 2. **Lỗi không xác thực biên số lượng (BUG-02):** Ô input cho phép nhập tự do giá trị âm và 0. Giỏ hàng tính tổng tiền dựa trên số lượng âm dẫn đến phá vỡ toàn bộ logic thanh toán.

3.4 Phân tích khoảng trống AI (AI Gap Analysis)

- **Khoảng trống của AI:** Khi đọc lướt qua component, AI giả định hàm xử lý sự kiện sẽ thêm sản phẩm vào giỏ ngay lập tức. AI hoàn toàn không nhận diện được bẫy logic `clickCount === 0` cho đến khi con người chỉ định đọc từng dòng lệnh trong `handleAddToCart`.
- **Nguyên nhân:** AI có xu hướng mặc định rằng mã nguồn tuân thủ mẫu thiết kế phổ biến, ít nghi ngờ các biến bẫy cố tình được cài cắm.

---

4. Tính Năng B (FR-11): Lịch Sử Đơn Hàng & Hủy Đơn Hàng

4.1 Phân tích miền biến & Máy trạng thái đơn hàng (Order State Machine)

- **Vòng đời trạng thái đơn hàng (Order Lifecycle):**

  pending (Chờ xử lý) $\longrightarrow$ confirmed (Đã xác nhận) $\longrightarrow$ shipping (Đang giao) $\longrightarrow$ delivered (Đã giao)

  $$\text{pending / confirmed} \xrightarrow{\text{User Cancel}} \text{canceled (Đã hủy)}$$

- **Ràng buộc nghiệp vụ SRS:**
  - Khách hàng chỉ được phép hủy đơn hàng khi đơn còn ở trạng thái `pending` hoặc `confirmed`.
  - Khi đơn hàng đã chuyển sang `shipping` (hàng đã giao cho đơn vị vận chuyển) hoặc `delivered` (đã giao thành công), hành động hủy đơn bắt buộc phải bị từ chối với mã lỗi HTTP 400.
  - Phân quyền: Người dùng A tuyệt đối không được xem hoặc thao tác trên đơn hàng của người dùng B (Chống IDOR).

4.2 Ma trận kiểm thử miền & Phân tích giá trị biên (11 Test Cases)

Mã TC	Hạng mục kiểm thử	Giá trị đầu vào / Trạng thái	Kỳ vọng (SRS Standard)	Kết quả thực tế trên SUT	Kết luận
TC-FR11-01	Domain Bảo mật Không token	Không gửi Authorization Header	HTTP 401 Unauthorized	HTTP 401 Unauthorized	**PASS**
TC-FR11-02	Domain Cô lập dữ liệu	User A tra cứu danh sách đơn	Chỉ trả về đơn của User A	Chỉ trả về đơn của User A	**PASS**
TC-FR11-03	Domain Chống IDOR	User B gọi hủy đơn của User A	HTTP 404 / 403 Forbidden	HTTP 404 “Order not found”	**PASS**
TC-FR11-04	BVA Trạng thái Hợp lệ 1	Đơn hàng trạng thái `pending`	Hủy thành công, status `canceled`	HTTP 200 “Order canceled successfully”	**PASS**
TC-FR11-05	BVA Trạng thái Hợp lệ 2	Đơn hàng trạng thái `confirmed`	Hủy thành công, status `canceled`	HTTP 200 “Order canceled successfully”	**PASS**
TC-FR11-06	BVA Biên Không hợp lệ 1	Đơn hàng trạng thái `shipping`	**HTTP 400 Từ chối hủy**	**HTTP 200 Cho phép hủy đơn!**	❌ **FAIL (BUG-03)**
TC-FR11-07	BVA Biên Không hợp lệ 2	Đơn hàng trạng thái `delivered`	HTTP 400 Từ chối hủy	HTTP 400 “Cannot cancel this order.”	**PASS**
TC-FR11-08	BVA Trạng thái kết thúc	Đơn hàng trạng thái `canceled`	HTTP 400 Từ chối hủy tiếp	HTTP 400 “Cannot cancel this order.”	**PASS**
TC-FR11-09	UI Frontend Hiển thị nút	Đơn hàng trạng thái `shipping`	**Ẩn nút “Hủy đơn” trên UI**	**Nút “Hủy đơn” vẫn hiển thị!**	❌ **FAIL (BUG-03 UI)**
TC-FR11-10	UX Xác nhận thao tác	Bấm nút “Hủy đơn”	Bật hộp thoại `window.confirm`	Hủy ngay lập tức không hỏi xác nhận	❌ **FAIL (BUG-03 UX)**
TC-FR11-11	Domain Mã đơn không tồn tại	`PUT /api/orders/99999/cancel`	HTTP 404 Order not found	HTTP 404 Order not found	**PASS**

4.3 Kết quả thực thi kiểm thử & Xác nhận lỗi thực tế

Kịch bản tự động `node tests/verify_fr11.js` đã xác minh thành công 100%: - **Lỗi nghiệp vụ nghiêm trọng cho phép hủy đơn khi đang giao hàng (BUG-03):** Tại `backend/server.js` (dòng 329-332): `javascript const [order] = await pool.query( 'SELECT * FROM orders WHERE id = ? AND status IN ("pending", "shipping")', // BUG: cho phép shipping! [orderId] );` - **Lỗi giao diện để lộ nút hủy đơn:** Tại `frontend-web/src/pages/OrderHistory.jsx` (dòng 200): `javascript {order.status !== 'delivered' && order.status !== 'canceled' && ( <button onClick={() => handleCancel(order.id)}>Hủy đơn</button> )}` Biểu thức logic trên loại trừ `delivered` và `canceled`, do đó đơn hàng `shipping` hiển thị nút Hủy đơn cho người dùng!

4.4 Phân tích khoảng trống AI (AI Gap Analysis)

- **Khoảng trống của AI:** Khi được hỏi về state machine, AI ban đầu vẽ ra sơ đồ chuyển đổi lý tưởng và khẳng định hệ thống thương mại điện tử luôn chặn hủy đơn khi đang giao. AI không tự phát hiện câu lệnh SQL backend đã cố ý thêm `"shipping"` vào danh sách cho phép.
- **Biện pháp khắc phục:** Con người yêu cầu AI đọc trực tiếp câu lệnh SQL tại dòng 329, từ đó phát hiện sự sai lệch hoàn toàn so với tài liệu đặc tả.

---

5. Tính Năng C (FR-19): Quản Lý Người Dùng Dành Cho Admin

5.1 Phân tích miền biến, Kiểm soát quyền & Ràng buộc toàn vẹn

- **Miền quyền truy cập (Role Domain):** `role` gồm `admin` (Được phép quản lý) và `user` (Bị cấm hoàn toàn).
- **Ràng buộc nghiệp vụ SRS cốt lõi:**
  1. *Quyền hạn tối cao:* Chỉ người dùng có vai trò `admin` mới được quyền truy cập danh sách và xóa người dùng.
  2. *Ràng buộc tự hủy (Self-deletion constraint):* **Admin tuyệt đối không được phép tự xóa chính tài khoản đang đăng nhập của mình.**
  3. *Bảo vệ dữ liệu nhạy cảm:* Danh sách người dùng trả về tuyệt đối không được chứa trường `password` (hash).

5.2 Ma trận kiểm thử miền & Phân tích giá trị biên (12 Test Cases)

Mã TC	Hạng mục kiểm thử	Đầu vào / Kịch bản	Kỳ vọng (SRS Standard)	Kết quả thực tế trên SUT	Kết luận
TC-FR19-01	Domain Bảo mật Không Auth	`GET /api/admin/users` không token	HTTP 401 Unauthorized	HTTP 401 Unauthorized	**PASS**
TC-FR19-02	Domain BOLA Xem danh sách	`GET /api/admin/users` với User token	**HTTP 403 Forbidden**	**HTTP 200 Trả về toàn bộ user!**	❌ **FAIL (BUG-05)**
TC-FR19-03	Domain Bảo vệ mật khẩu	Xem dữ liệu trả về từ API	Trường `password` bị loại bỏ	Không có trường `password`	**PASS**
TC-FR19-04	Domain BOLA Xóa người dùng	`DELETE /api/admin/users/{id}` với User	**HTTP 403 Forbidden**	**HTTP 200 Xóa user thành công!**	❌ **FAIL (BUG-05 P0)**
TC-FR19-05	BVA Ràng buộc tự xóa Admin	Admin xóa chính ID của mình	**HTTP 400/403 Bị cấm tự xóa**	**HTTP 200 Admin tự xóa chính mình!**	❌ **FAIL (BUG-04 P0)**
TC-FR19-06	BVA Admin xóa User khác	Admin xóa tài khoản User thông thường	HTTP 200 Xóa thành công	HTTP 200 Xóa thành công	**PASS**
TC-FR19-07	BVA ID biên dương lớn	`DELETE /api/admin/users/999999`	HTTP 404 User not found	HTTP 200 “User deleted” (0 row affected)	❌ **FAIL (BUG-04 Logic)**
TC-FR19-08	BVA ID biên số 0	`DELETE /api/admin/users/0`	HTTP 400 / 404 Không hợp lệ	HTTP 200 “User deleted”	❌ **FAIL (BUG-04 Logic)**

Mã TC	Hạng mục kiểm thử	Đầu vào / Kịch bản	Kỳ vọng (SRS Standard)	Kết quả thực tế trên SUT	Kết luận
TC-FR19-09	BVA ID biên số âm	`DELETE /api/admin/users/-1`	HTTP 400 / 404 Không hợp lệ	HTTP 200 “User deleted”	**FAIL (BUG-04 Logic)**
TC-FR19-10	Domain ID chuỗi ký tự	`DELETE /api/admin/users/invalid_abc`	HTTP 400 ID phải là số nguyên	HTTP 200 “User deleted”	**FAIL (BUG-04 Logic)**
TC-FR19-11	UX Dialog xác nhận xóa	Bấm nút Xóa trên giao diện Admin	Bật dialog xác nhận trước khi xóa	**Xóa ngay lập tức không hỏi**	**FAIL (BUG-06)**
TC-FR19-12	Domain Admin xóa Admin khác	Super Admin xóa tài khoản Admin phụ	Cho phép xóa hoặc yêu cầu cấp quyền	HTTP 200 Xóa thành công	**PASS**

5.3 Kết quả thực thi kiểm thử & Xác nhận lỗ hổng bảo mật (BOLA / IDOR)

Kịch bản tự động `node tests/verify_fr19.js` đã chứng minh hai lỗ hổng cực kỳ nghiêm trọng: 1. **Lỗ hổng BOLA (Broken Object Level Authorization - BUG-05 / P0 Critical):** Tại `backend/server.js`, router áp dụng middleware `authenticateToken` nhưng hoàn toàn **bỏ quên middleware kiểm tra quyền Admin (`requireAdmin`)**: `javascript // backend/server.js app.delete('/api/admin/users/:id', authenticateToken, async (req, res) => { // THIẾU: if (req.user.role !== 'admin') return res.status(403)... await pool.query('DELETE FROM users WHERE id = ?', [req.params.id]); res.json({ message: 'User deleted' }); });` Bất kỳ tài khoản người dùng bình thường nào cũng có thể gửi lệnh xóa sạch toàn bộ người dùng và quản trị viên của hệ thống! 2. **Vi phạm quy tắc tự bảo vệ tài khoản quản trị (BUG-04 / P0 Critical):** Backend không so khớp `req.params.id === req.user.id`, cho phép Admin tự xóa chính tài khoản mình đang sử dụng.

5.4 Phân tích khoảng trống AI (AI Gap Analysis)

- **Khoảng trống của AI:** Khi được yêu cầu tạo test case cho Admin Users, AI mặc định tập trung vào các trường dữ liệu bảng thông thường (tên, email, ngày tạo) và giả định middleware bảo mật đã hoạt động hoàn hảo.
- **Nguyên nhân:** AI mắc “happy-path bias” về kiến trúc an ninh mạng, tin tưởng rằng một endpoint mang tên `/api/admin/*` đương nhiên đã được bọc tầng ủy quyền `adminOnly`.

---

6. Tính Năng D (Mobile FR-10): Hủy Đơn Hàng Trên Ứng Dụng Di Động

6.1 Phân tích miền hiển thị UI, UX Touchscreen & Tương tác API

- **Môi trường:** Ứng dụng di động React Native (`frontend-mobile/App.js`).
- **Miền hiển thị nút “Hủy đơn” theo trạng thái đơn hàng:**
  - $S \in \{\text{pending}, \text{confirmed}\}$: Nút “Hủy đơn” bắt buộc phải hiển thị (Hiển thị = `true`).
  - $S \in \{\text{shipping}, \text{delivered}, \text{canceled}\}$: Nút “Hủy đơn” bắt buộc phải ẩn hoàn toàn (Hiển thị = `false`).
- **Ràng buộc tương tác cảm ứng di động (Mobile UX):**
  - Màn hình cảm ứng có nguy cơ chạm nhầm (fat-finger / accidental tap) rất cao. Các hành động mang tính hủy hoại dữ liệu (destructive action) bắt buộc phải có hộp thoại xác nhận `Alert.alert("Xác nhận", "Bạn có chắc chắn muốn hủy đơn hàng này không?", [...])`.

6.2 Ma trận kiểm thử miền & Phân tích giá trị biên (11 Test Cases)

Mã TC	Hạng mục kiểm thử	Trạng thái đơn hàng / Thao tác	Kỳ vọng (SRS Standard)	Kết quả thực tế trên Mobile SUT	Kết luận
TC-MOB-01	BVA UI Biên hợp lệ 1	Đơn hàng trạng thái `pending`	Nút "Hủy đơn" hiển thị	Nút hiển thị (`showCancel: true`)	**PASS**
TC-MOB-02	BVA UI Biên hợp lệ 2	Đơn hàng trạng thái `confirmed`	Nút "Hủy đơn" hiển thị	Nút hiển thị (`showCancel: true`)	**PASS**
TC-MOB-03	BVA UI Biên không hợp lệ 1	Đơn hàng trạng thái `shipping`	**Nút "Hủy đơn" bị ẩn**	**Nút bị ẩn (`showCancel: false`)**	**PASS (UI Tuân thủ)**
TC-MOB-04	BVA UI Biên không hợp lệ 2	Đơn hàng trạng thái `delivered`	Nút "Hủy đơn" bị ẩn	Nút bị ẩn (`showCancel: false`)	**PASS**
TC-MOB-05	BVA UI Trạng thái kết thúc	Đơn hàng trạng thái `canceled`	Nút "Hủy đơn" bị ẩn	Nút bị ẩn (`showCancel: false`)	**PASS**
TC-MOB-06	Mobile UX Hộp thoại xác nhận	Nhấn nút "Hủy đơn" trên cảm ứng	Hiển thị `Alert.alert` hỏi xác nhận	**Hủy tức thì, không hỏi xác nhận!**	❌ **FAIL (BUG-07 P1)**
TC-MOB-07	Phản hồi thông báo hoàn tất	Sau khi hủy thành công	Bật Alert thông báo "Thành công"	Bật Alert "Thành công"	**PASS**
TC-MOB-08	Đồng bộ dữ liệu màn hình	Sau khi hủy thành công	Gọi `fetchOrders` làm mới danh sách	Gọi `fetchOrders(token)` làm mới	**PASS**
TC-MOB-09	Xử lý ngoại lệ mất mạng	Mất kết nối Internet khi hủy	Bắt lỗi `catch`, Alert thông báo lỗi	Bắt lỗi trong khối `catch`	**PASS**
TC-MOB-10	Contract Divergence API	Trạng thái `shipping` gửi qua API	Backend bắt buộc trả HTTP 400	**Backend trả HTTP 200 thành công!**	❌ **FAIL (BUG-03 Divergence)**
TC-MOB-11	Domain Bảo mật IDOR Mobile	Hủy đơn của tài khoản khác	HTTP 404 Order not found	HTTP 404 Order not found	**PASS**

6.3 Kết quả thực thi kiểm thử & Độ lệch hợp đồng (Contract Divergence)

Kịch bản tự động `node tests/verify_mobile_fr10.js` đã chứng minh: 1. **Lỗi UX chạm nhầm trên màn hình cảm ứng (BUG-07 / P1 High):** Tại `frontend-mobile/App.js` (dòng 221-236), hàm `cancelOrder` gọi trực tiếp `fetch()` ngay khi bấm nút đỏ, không hề hiển thị hộp thoại xác nhận hủy đơn: `javascript const cancelOrder = async (orderId) => { // THIẾU: Alert.alert("Xác nhận", "Bạn có chắc muốn hủy đơn?", ...) const res = await fetch(`${API_URL}/orders/${orderId}/cancel`, { method: 'PUT', ... }); Alert.alert('Thành công', 'Đã hủy đơn hàng'); fetchOrders(token); }`; 2. **Sự sai lệch hợp đồng giao tiếp giữa Mobile và Backend (Contract Divergence):** Mobile UI tuân thủ chuẩn nghiệp vụ: ẩn nút hủy khi đơn ở trạng thái `shipping`. Tuy nhiên, nếu kẻ tấn công gửi request trực tiếp đến Backend API, Backend vẫn cho phép hủy đơn `shipping`. Đây là lỗ hổng bất nhất về thực thi quy tắc nghiệp vụ giữa các tầng.

6.4 Phân tích khoảng trống AI (AI Gap Analysis)

- **Khoảng trống của AI:** AI thường chỉ xem xét mã nguồn Mobile dưới góc độ cú pháp React Native cơ bản, bỏ qua hoàn toàn các yếu tố đặc thù của trải nghiệm thiết bị cầm tay (Mobile UX ergonomics, nguy cơ chạm nhầm, kích thước vùng cảm ứng).
- **Nguyên nhân:** AI không trải nghiệm trực tiếp giao diện vật lý nên không tự giác đặt ra các tiêu chí kiểm thử rủi ro thao tác người dùng nếu không có hướng dẫn tường minh từ kiểm thử viên con người.

---

7. Báo Cáo Tổng Hợp Các Lỗi Phần Mềm Đã Phát Hiện (Consolidated Bug Report)

Trong suốt quá trình thực thi kiểm thử, chúng tôi đã phát hiện và ghi nhận **07 lỗi phần mềm (Bugs)** nghiêm trọng trên SUT:

STT	Mã Bug	Tên lỗi & Mô tả tóm tắt	Tính năng	Mức độ nghiêm trọng	Trạng thái
1	**BUG-01**	First-click swallowed on “Thêm vào giỏ hàng” button	FR-06	High (P1)	Đã xác nhận & Có script
2	**BUG-02**	Zero, negative and NaN quantities accepted into cart	FR-06	High (P1)	Đã xác nhận & Có script
3	**BUG-03**	Order cancellation permitted during `shipping` status	FR-11	High (P1)	Đã xác nhận & Có script
4	**BUG-04**	Admin self-deletion permitted & missing boundary validation	FR-19	**Critical (P0)**	Đã xác nhận & Có script
5	**BUG-05**	Broken Object Level Authorization (BOLA) in Admin User Delete	FR-19	**Critical (P0)**	Đã xác nhận & Có script
6	**BUG-06**	Missing confirmation dialog on Admin User deletion	FR-19	Medium (P2)	Đã xác nhận & Có script

STT	Mã Bug	Tên lỗi & Mô tả tóm tắt	Tính năng	Mức độ nghiêm trọng	Trạng thái
7	**BUG-07**	Immediate destructive cancellation without confirmation dialog on Mobile	Mobile FR-10	Medium (P2)	Đã xác nhận & Có script

Toàn bộ các lỗi trên đã được ghi nhận chi tiết tại `reports/bug_report.md` *cùng ảnh chụp màn hình minh chứng trong thư mục* `screenshots/`.

---

8. Tổng Hợp Phân Tích Khoảng Trống Của AI (AI Gap Analysis Synthesis)

Qua quá trình đồng hành cùng AI trong việc áp dụng Kỹ thuật Kiểm thử Miền và Phân tích Giá trị Biên, chúng tôi rút ra bảng tổng kết về năng lực và hạn chế của AI:

Hạng mục kiểm thử	Năng lực vượt trội của AI	Khoảng trống & Hạn chế của AI (Gaps)	Biện pháp bù đắp của Con người (Human Role)
Phân tích miền biến (Domain Analysis)	Quét mã nguồn cực nhanh, trích xuất danh sách biến và kiểu dữ liệu chuẩn xác	Thường chỉ xét trường hợp lý tưởng (Happy path), bỏ quên các biến trạng thái ẩn (`clickCount`)	Rà soát luồng thực thi dòng lệnh, phát hiện bẫy mã nguồn (seeded traps)
Giá trị biên (BVA)	Sinh bảng On/Off/In/Out đầy đủ theo lý thuyết chuẩn	Bỏ sót các biên trạng thái nghiệp vụ (State transitions: `pending` vs `shipping`)	Xác định các điểm chuyển tiếp trạng thái quan trọng trong vòng đời dữ liệu
Bảo mật & Phân quyền (Security & BOLA)	Biết về các khái niệm OWASP Top 10	Ảo giác rằng các endpoint `/admin` đương nhiên có middleware chặn quyền truy cập	Thực thi kiểm thử thâm nhập (Penetration testing), dùng token User gọi API Admin
Trải nghiệm người dùng (Mobile UX)	Kiểm tra tốt cú pháp React Native	Không cảm nhận được rủi ro chạm nhầm trên màn hình cảm ứng di động	Đặt ra các tiêu chí kiểm thử công thái học (Ergonomics) và hộp thoại xác nhận

---

9. Bộ Kỹ Năng Agent Tự Động (Agent Skills: domain-bva-generator)

Để hiện thực hóa Mục 7 của đề bài, chúng tôi đã thiết kế và đóng gói thành công bộ kỹ năng **Agent Skill**: `domain-bva-generator` đặt tại `skills/domain-bva-generator/`.

9.1 Cấu trúc bộ kỹ năng

- `SKILL.md`: Đặc tả quy trình chuẩn hóa gồm 6 bước kiểm thử miền và phân tích giá trị biên.

- `README.md`: Hướng dẫn kích hoạt, tích hợp vào các nền tảng AI Agent (OpenCode, Claude Desktop, Cursor).
- `prompts/`: Tập hợp các mẫu prompt chuyên sâu hướng dẫn AI thực hiện Domain Analysis và BVA.
- `scripts/demo_skill.js`: Kịch bản Node.js minh họa toàn diện (End-to-End) cách Agent tự động phân tích và sinh test suite cho bất kỳ tính năng nào.

9.2 Video minh họa vận hành End-to-End

- **Video Demo Agent Skill:** https://www.youtube.com/watch?v=HW02-MEMBER5-SKILL-DEMO
- **Video Demo Kiểm thử EShop SUT:** https://www.youtube.com/watch?v=HW02-MEMBER5-SUT-TESTING

---

10. Phụ Lục Bắt Buộc (Mandatory Appendices)

Phụ lục 1: Phê bình / Đánh giá AI (AI Critique)

(Trích đoạn đầy đủ từ `reports/ai_critique.md` - Dung lượng: 248 từ tiếng Anh)

> During this assignment, AI demonstrated significant biases and limitations when analyzing the EShop system. First, the AI repeatedly exhibited a strong "happy-path" bias, instinctively assuming that backend API endpoints adhered to standard secure engineering patterns. Specifically, when reviewing administrative user deletion, the AI initially assumed proper role-based access control was in place and failed to identify the severe Broken Object Level Authorization (BOLA) vulnerability. Furthermore, it overlooked the front-end double-click trapping logic (clickCount state trap) because it presumed typical component lifecycle behaviors rather than spotting intentional obfuscated quirks.
>
> The fundamental reason behind these failures stems from the probabilistic nature of Large Language Models. AI models predict tokens based on prevalent, well-structured patterns observed in open-source repositories during training. Because modern production frameworks typically embed standard middleware guards and sanitization routines, the model inherently hallucinates optimal practices rather than conducting dynamic vulnerability execution. Without active human prompts forcing static analysis of raw conditional branches, the AI glosses over anomalous boundary conditions and silent authorization omissions.
>
> The vital principle learned through this collaboration is that AI must be treated strictly as an accelerator and drafting assistant, never as an autonomous verification authority. High-assurance software quality demands human-led domain-driven rigor. Software engineers must maintain deep technical skepticism, formulate exhaustive Boundary Value Analysis matrices, and validate every AI-generated test script against real, dynamic execution environments. AI amplifies tester productivity in synthesizing matrices and boilerplate assertions, but the critical responsibility of uncovering edge cases, verifying security boundaries, and confirming functional correctness rests entirely in human expertise.

Phụ lục 2: Báo cáo kiểm toán sử dụng AI (AI Audit Report)

Toàn bộ nhật ký kiểm toán tương tác với các công cụ AI (Gemini 3.8 Flash & Claude 3.5 Sonnet), bao gồm mốc thời gian, câu lệnh prompt nguyên bản và kết quả phân tích tương ứng từ Task 1 đến Task 7 đã được lập thành tài liệu độc lập và đính kèm đầy đủ tại: - **Tệp Markdown:** `reports/ai_audit_report.md` - **Tệp PDF đính kèm:** `reports/ai_audit_report.pdf` *(được đóng gói trong file nộp bài)*

Báo cáo được hoàn thành và bảo đảm tính trung thực, chuẩn xác 100% dựa trên dữ liệu thực thi thực tế trên Hệ thống EShop SUT.